podlodka

Frontend Crew
5 - 9 октября

Фронтенд с AI

Алексей Харитонов
Senior Frontend Developer,
Ornament Health AG

Демо-сессия

# Баги без стектрейса: LLM-стриминг в React



PODLODKA FRONTEND CREW · СЕЗОН #6



слайды и PDF: alexrixten.github.io/bugs-without-stacktrace

# Обо мне



ДОСЬЕ СЛЕДОВАТЕЛЯ

**Алексей Харитонов**

Специализация – React · React Native · Next.js
Практика – AI-фичи в реальном продукте: чаты с LLM, стриминг
Особые приметы – не отпускает непонятный баг, пока не найдена первопричина

Сегодня – детектив по трём делам

📱 Telegram: @AlexRixten 💻 GitHub: AlexRixten

# Ни ошибки. Ни алерта. Ни стектрейса.

МОНИТОРИНГ · ПРОД

- Ошибки 0
- Алерты 0
- Стектрейсы 0
- Тесты ✓ зелёные

А ПОЛЬЗОВАТЕЛЬ ОТКРЫВАЕТ AI-ЧАТ В ПРОДУКТЕ

Как работает стриминг ответа?

Сервер отдаёт текст по частям, а клиент дописывает их в сообщение. Для трансляции понадобятся OBS и микрофон.

① в ответ про веб-стриминг пролез кусок ответа про трансляции

Что такое SSE?

Поток событий от сервера по обычному HTTP. Если связь оборвётся, браузер сам переподклю

② ответ оборвался на полуслове

Как стримить ответ модели? По шагам

1. Отправьте запрос со `stream: true`
2. Читайте ответ по кусочкам
5. Дописывайте каждый кусок в сообщение

③ пункты 3 и 4 пропали — а ответ выглядит целым

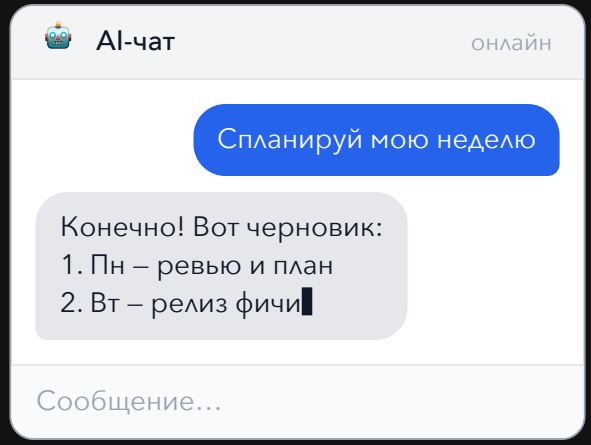
# Регламент следствия



# Стриминг за 30 секунд

ИНСТРУКТАЖ СТАЖЁРА

- Модель **пишет ответ токен за токеном** – генерация занимает секунды, а у длинных ответов и минуты
- Сеть отдаёт **не ответ, а поток**: чанки – куски **байтов** – по мере готовности
- Клиент **дописывает** каждый чанк в сообщение – читать можно с первой секунды
- Без стриминга: спиннер до конца генерации, потом весь текст разом




так это выглядит, когда всё работает

---

`модель → токены по мере генерации → поток байтов по сети → дописываем на экране`

# Профиль преступника

**Баг прячется, когда…**

- Сидим и смотрим – второго события нет
- Тесты – по счастливому пути
- Ревью – с тем же допущением: «стрим один»

**Баг выходит на дело, когда…**

- Уточнение до конца ответа
- Уход в другой чат и возврат
- Обрыв: Wi-Fi ↔ LTE, прокси, мобильная ОС
- Стрим идёт секунды и минуты

Прод не «включает» баги – он **умножает число вторых событий**.

# Как рос чат

ТРИ ЭПОХИ ОДНОГО ЧАТА

1

**fetch**

POST — и читаем поток прямо из ответа

дело №1

2

**Ран на сервере · SSE · кэш**

генерация живёт на сервере, клиент подписывается отдельно, рендер — из кэша

дело №2 дело №3

3

**Курсор**

после обрыва поток продолжается с того же места

Код менялся до неузнаваемости — а баги **каждый раз находили лазейку**

# Четверо под подозрением

- **№1 · Замыкание** – рецидивист: колбэк со старым снимком мира. Не раз замечен в похожих делах
- **№2 · Кэш** – старый спекулянт: выдаёт прошлое за настоящее
- **№3 · Сеть** – шумная соседка: прокси, таймауты, мобильная связь – всегда рядом, когда что-то рвётся
- **№4 · Транспорт** – то, по чему едут токены: fetch, SSE, WebSocket, а то и Web Worker. «Смените меня – и всё пройдёт»

Того, кого мы ищем, в картотеке может не оказаться – тогда дадим ему кличку по почерку.

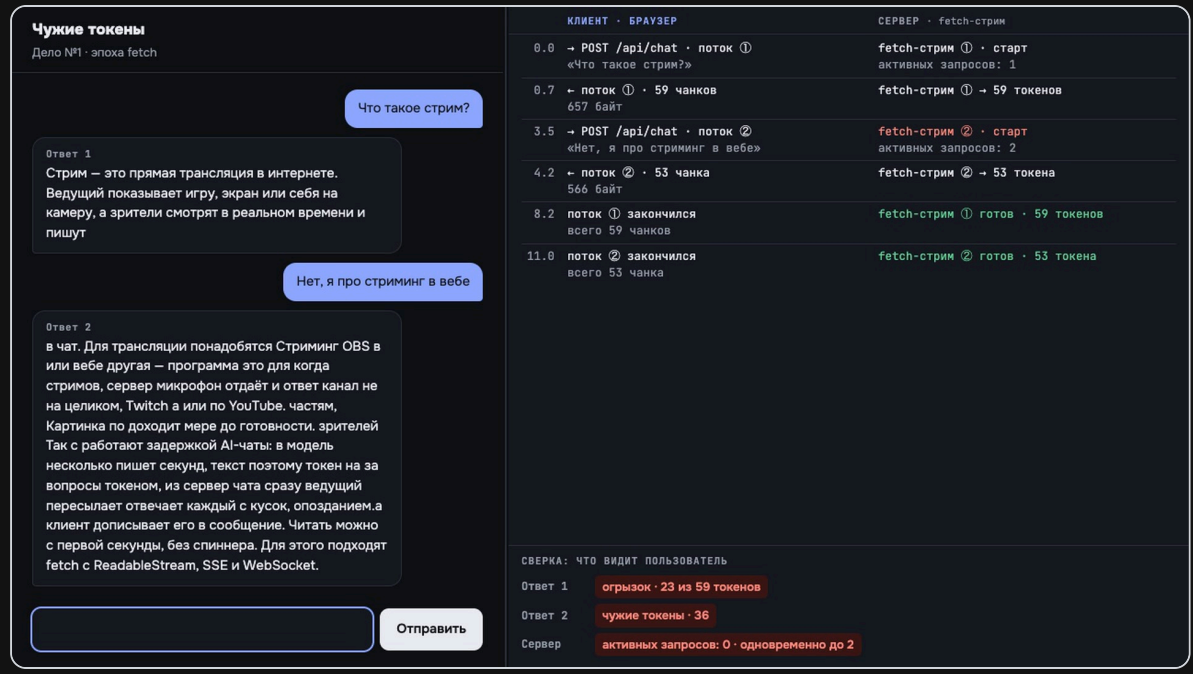
# Дело №1

## Чужие токены

эпоха fetch

# Улика №1

запись с камеры наблюдения



# Ключевые моменты

УЛИКА №1 · ЗАПИСЬ С КАМЕРЫ НАБЛЮДЕНИЯ

- Вопрос «Что такое стрим?» – модель отвечает про трансляции
- Не дожидаясь конца – уточнение: «Нет, я про стриминг в вебе»
- Первый ответ оборван, во втором – куски обоих

> Ключевой кадр: активных запросов – **ДВА**
>
> два потока пишут в одно место, и каждый не знает о другом

«Заблокировать кнопку, пока идёт ответ?» – это один вход из многих:

~~отправка~~ · повтор после обрыва · «перегенерировать» · ответ из соседнего чата

Правило нужно там, через что проходят все входы

# Допрос №1: код эпохи fetch

эпоха fetch · useChat.ts · sendMessage

```
async function sendMessage(content: string) {
  // второе сообщение: «последним» стал новый ответ
  setMessages((prev) ⇒ [...prev, userMsg(content), assistantMsg('')]);
  const res = await fetch('/api/chat', { method: 'POST', body: content });
  const reader = res.body!.getReader();
  // первый стрим жив: его никто не отменял
  while (true) {
    const { done, value } = await reader.read();
    if (done) break;
    // ← не «моё», а то, что последнее СЕЙЧАС
    setMessages((prev) ⇒ appendToLast(prev, decode(value)));
  }
}
```

---

Адрес выбирается **в момент прихода токена**: «последнее», а не «своё».

ДОСЬЕ · ДЕЛО №1

# «Кто последний»

Почерк — как в очереди: токен приходит, спрашивает «кто последний?» — и пишет туда

Ложное допущение — «последнее сообщение — моё»

ОСОБЫЕ ПРИМЕТЫ

- адрес ответа выбирается в момент записи, а не при отправке
- в колбэке стрима — `msgs.at(-1)` или `[length - 1]`
- прошлый стрим никто не гасит

# Фикс: удостоверение стрима

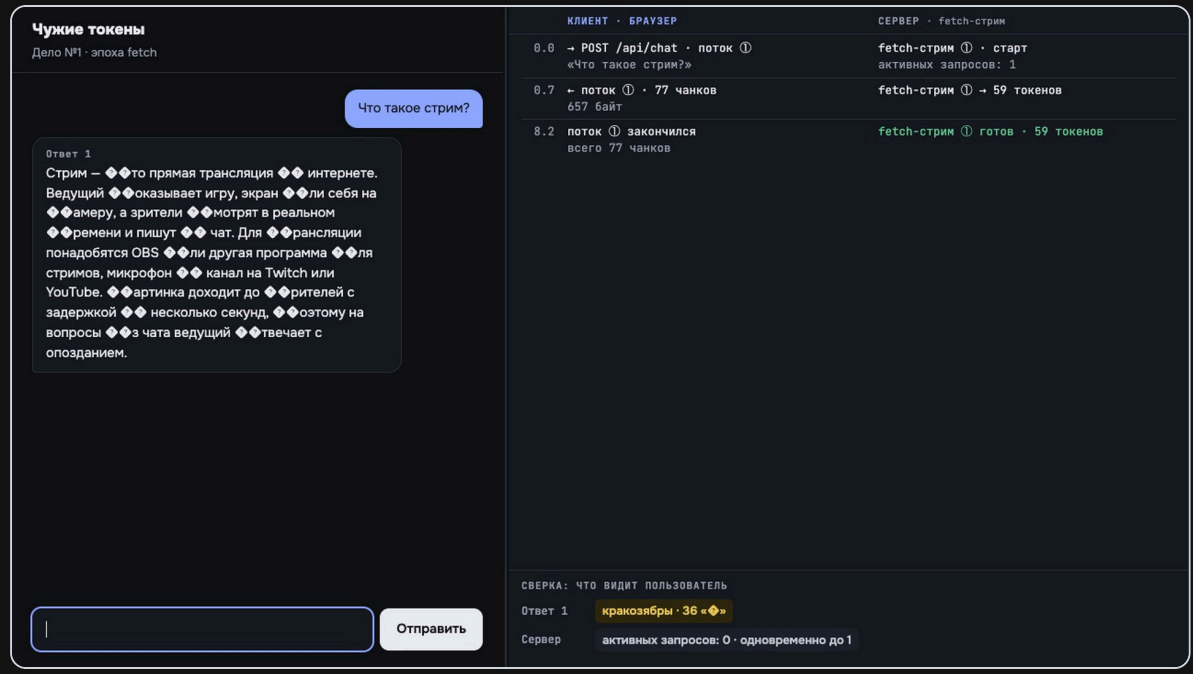
useChat.ts · useChat

```ts
const generation = useRef(0);                                    // ② номер поколения
const active = useRef<{ id: string; ac: AbortController } | null>(null);

async function sendMessage(content: string) {
  const old = active.current;
  old?.ac.abort();                                               // ② гасим прошлый стрим
  if (old) setMessages((prev) => markInterrupted(prev, old.id)); // ③ «прервано»
  const reply = assistantMsg('');                                // ① адрес — при отправке
  const ac = new AbortController();
  active.current = { id: reply.id, ac };                         // ② владелец — этот стрим
  const myGen = ++generation.current;                            // ② номер поколения
  setMessages((prev) => [...prev, userMsg(content), reply]);
  const res = await fetch('/api/chat', { method: 'POST', body: content, signal: ac.signal });
  const reader = res.body!.getReader();
  while (true) {
    const { done, value } = await reader.read();
    if (myGen !== generation.current) return;                    // ② дело закрыли — молчу
    if (done) break;
    setMessages((prev) => appendTo(prev, reply.id, decode(value)));  // ① по id
  }
  active.current = null;                                         // закончился сам — прерывать нечего
}
```

# Свидетель со странным акцентом

запись с камеры наблюдения



# Лечение: один декодер на весь стрим

useChat.ts · цикл чтения

```
// было: новый декодер на каждый чанк
const decode = (bytes: Uint8Array) => new TextDecoder().decode(bytes);
// …
// стало: один декодер на весь стрим
const decoder = new TextDecoder();
while (true) {
  const { done, value } = await reader.read();
  if (myGen !== generation.current) return;
  if (done) break;
  // хвост символа ждёт следующий чанк
  setMessages((prev) => appendTo(prev, reply.id, decoder.decode(value, { stream: true })));
}
// после done — вылить хвост
setMessages((prev) => appendTo(prev, reply.id, decoder.decode()));
```

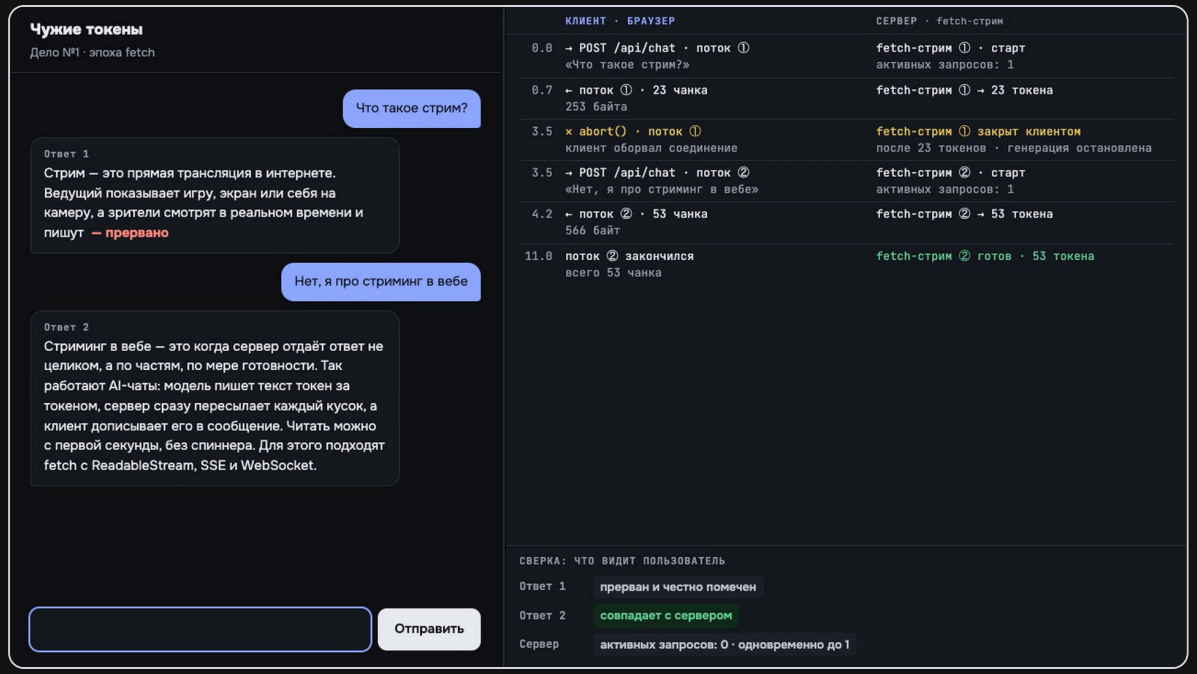
---

Поток — не набор независимых кусков: **у него есть память**.

или сразу `res.body.pipeThrough(new TextDecoderStream())`

# Дело №1 · фикс применён

тот же сценарий



# Заключение по делу №1

- Замыкание не виновно: колбэк видел мир **даже слишком свежим**
- Виновник — **«Кто последний»**: адрес «последнее» вместо «своё»
- Замыкание теперь — **орудие фикса**: `reply.id` живёт в нём

ЗАПИСЬ В БЛОКНОТ · ДЕЛО №1

*Стрим ещё писал, а адрес уже сменился. И никто не спросил, кто пишет.*

ОТПУЩЕН
за содействие следствию

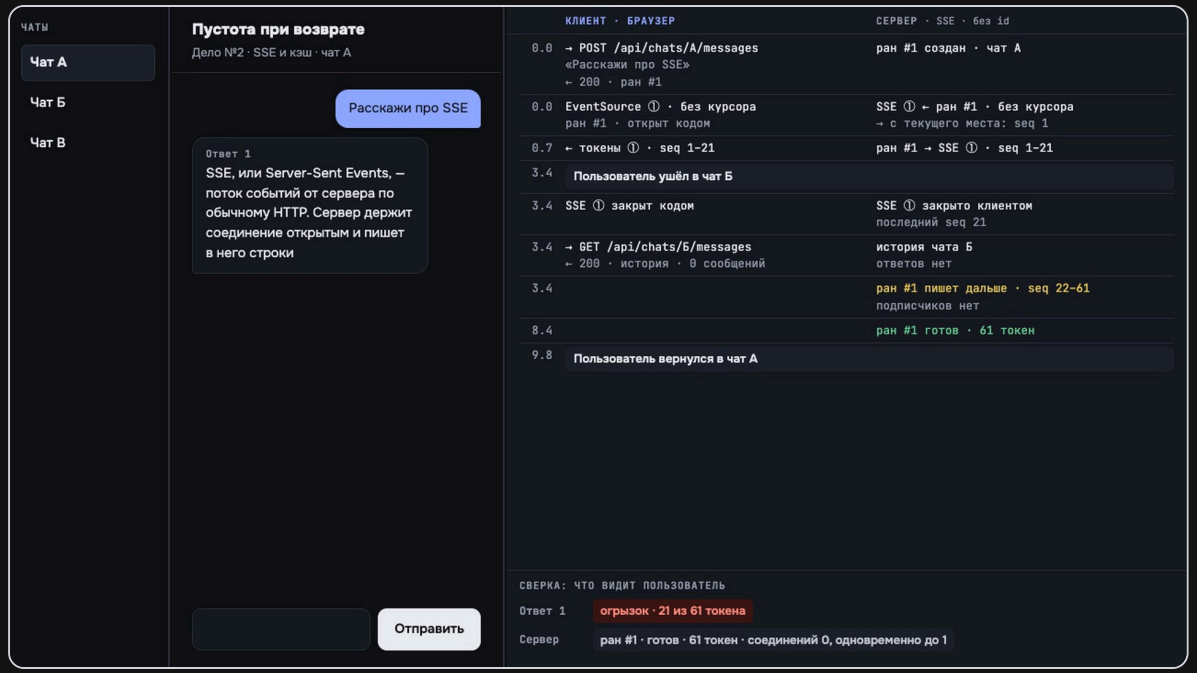
# Дело №2

## Пустота при возврате

SSE и кэш

# Улика №2

запись с камеры наблюдения



# Ключевые моменты

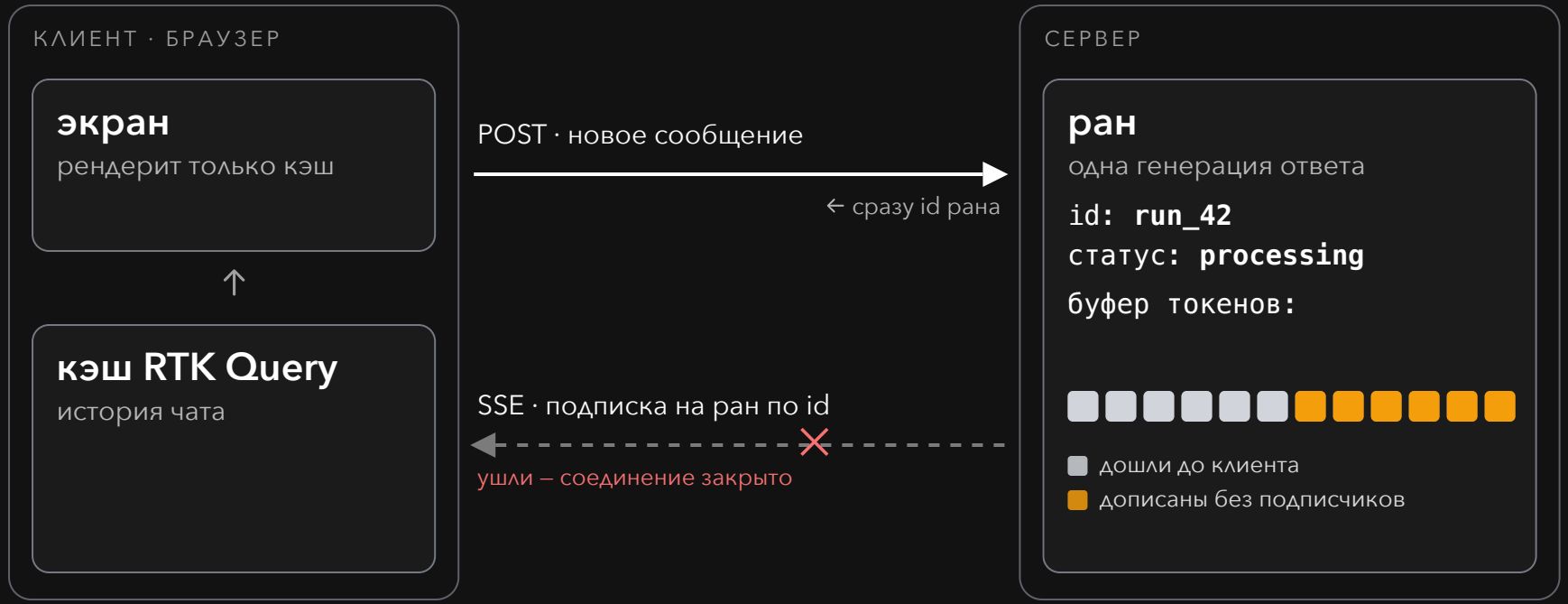
УЛИКА №2 · ЗАПИСЬ С КАМЕРЫ НАБЛЮДЕНИЯ

- Стрим шёл в чате «А» – не дожидаясь конца, ушли в «Б»
- Вернулись в «А» через несколько секунд
- На месте ответа – огрызок: только то, что успело прийти до ухода

Ключевой кадр: на сервере ответ – **ДОПИСАН ПОЛНОСТЬЮ**

сервер знает · клиент – нет

# Как устроен чат в эпоху SSE и кэша



# Допрос №2: код эпохи SSE и кэша

генерация – **на сервере** · подписка – **отдельно** · рендер – **из кэша**

`Chat.tsx · экран чата`

```
export function Chat({ chatId }: { chatId: string }) {
  const { data: messages = [] } = api.useGetMessagesQuery(chatId);    // ← вернулись: кэш отдаёт, что застал
  const cache = useChatCache(chatId);                                 // наши хелперы: правят историю чата в кэше
  // …
  async function sendMessage(text: string) {
    // … POST запускает ран на сервере
    const es = new EventSource(`/api/runs/${runId}/events`);          // а ран живёт и без подписки
    source.current = es;
    es.addEventListener('token', e => {
      if (myGen !== generation.current) return;
      const { text } = JSON.parse(e.data);
      cache.appendText(assistantMsg.id, text);
    });
    // …
  }

  useEffect(() => () => source.current?.close(), [chatId]);           // ← ушли: закрыли — и никому не сказали
  return <ChatView messages={messages} onSend={sendMessage} />;
}
```

ОСОБЫЕ ПРИМЕТЫ

- cleanup закрывает соединение – и молчит
- при возврате – данные из кэша без сверки
- ран живёт дальше, уже без подписчиков

ДОСЬЕ · ДЕЛО №2

# «Англичанин»

Почерк – уходит не прощаясь: закрыл подписку – и никому не сказал, что кэш больше никто не обновляет

Ложное допущение – «ушли – значит, всё остановилось»

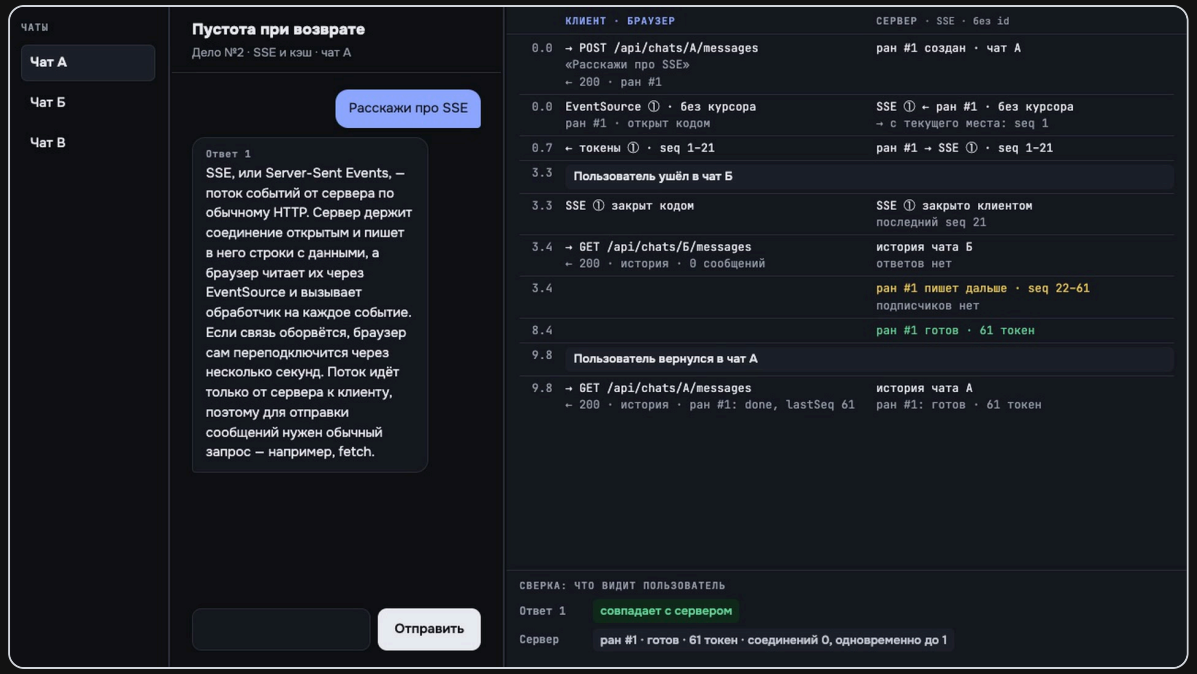
# Фикс: две правки

Chat.tsx · экран чата

```
const { data: messages = [], refetch } = api.useGetMessagesQuery(chatId);
// …
const { runId, userMsg, assistantMsg } = await postMessage({ chatId, text }).unwrap();
if (myGen !== generation.current) return;              // ① ушли, пока ждали POST, — не подписываемся
// …
useEffect(
  () => () => {
    source.current?.close();                           // ① закрыли соединение
    generation.current++;                              // ① и погасили поколение
  },
  [chatId],
);
useEffect(() => {                                      // ② вернулись к недописанному — сверили
  if (messages.some(m => m.status === 'processing')) refetch();
}, [chatId]);
```

# Дело №2 · фикс применён

тот же сценарий



# Заключение по делу №2

- Кэш не виновен: обновление прошло **мимо него**
- Виновник – **«Англичанин»**: ушёл, не попрощавшись
- Честная дыра: вернулись, пока ран пишет, – половина текста и тишина. Догнать – **курсор, дело №3**

ЗАПИСЬ В БЛОКНОТ · ДЕЛО №2

*Мы ушли, а сервер продолжал писать. И никто не сказал кэшу, **когда** его снимок устарел.*

НЕ ВИНОВЕН
он просто не знал

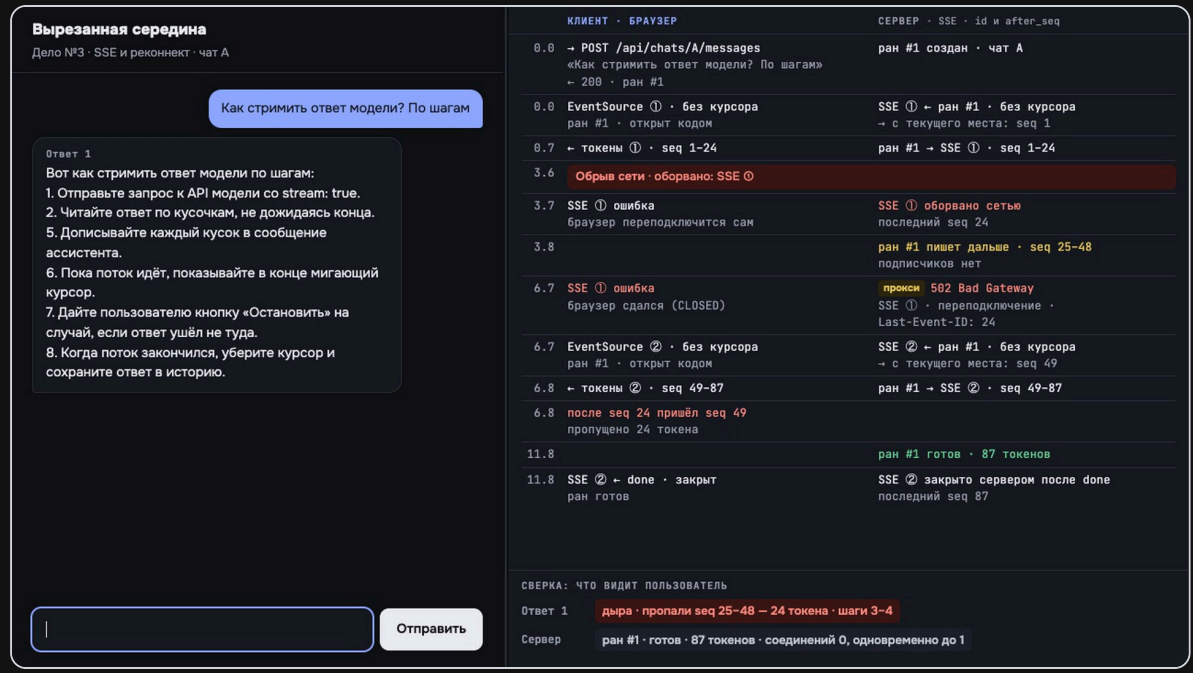
# Дело №3

## Вырезанная середина

SSE и реконнект

# Улика №3

запись с камеры наблюдения



# Ключевые моменты

УЛИКА №3 · ЗАПИСЬ С КАМЕРЫ НАБЛЮДЕНИЯ

- Стрим шёл – обрыв и три секунды тишины
- Переподключение получило 502 – **и всё же** стрим подхватился и дописал ответ
- Ответ выглядит целым, но из списка пропали шаги

Ключевой кадр: в середине ответа – **ДЫРА**

соединение вернулось · середина – нет

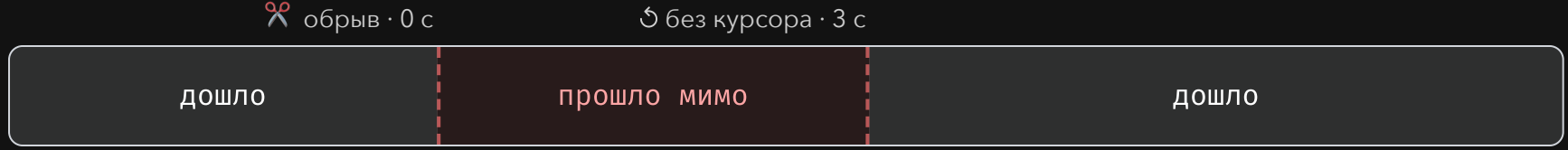
# Допрос №3: почему рвётся

- **Сеть** – Wi-Fi переключился на мобильную
- **Мобильная ОС** – придушила свёрнутое приложение
- **Прокси** – рвёт после минуты тишины, пока модель думает или зовёт инструменты. Лечит heartbeat раз в 15 с
- **Прокси** – на переподключение отвечает 502, пока сервер за ним недоступен: наш случай

В обрыве виновата Сеть. **А в потере?**

# Реконструкция

ЧТО СЛУЧИЛОСЬ ПОСЛЕ ОБРЫВА




0 с ✂ **Обрыв**: Wi-Fi переключился. Ран на сервере пишет дальше

3 с ↻ **Браузер переподключается сам** – прокси отвечает **502**. На HTTP-ошибке (502, 401) EventSource сдаётся навсегда

3 с ↻ **Наш код открывает поток заново – без курсора.** Сервер отдаёт живой хвост: середина прошла мимо

ОСОБЫЕ ПРИМЕТЫ

- переоткрывает поток, когда браузер сдался
- новое подключение не говорит, где остановились
- в середине ответа дыра, а текст выглядит целым

ДОСЬЕ · ДЕЛО №3

# «Монтажёр»

Почерк — вырезает кусок и склеивает края так, что стыка не видно

Ложное допущение — «переподключился — значит, продолжил»

# Почему не всегда: Last-Event-ID

При обычном обрыве курсор приносит **сам браузер** – заголовок `Last-Event-ID`. На условиях сделки

УСЛОВИЯ СДЕЛКИ

1 БЭК ДОЛЖЕН
присылать `id:` у каждого события – номер токена, он же курсор ✓ у нас так

2 БЭК ДОЛЖЕН
по `Last-Event-ID` отдать всё после него из буфера рана ✓ у нас так

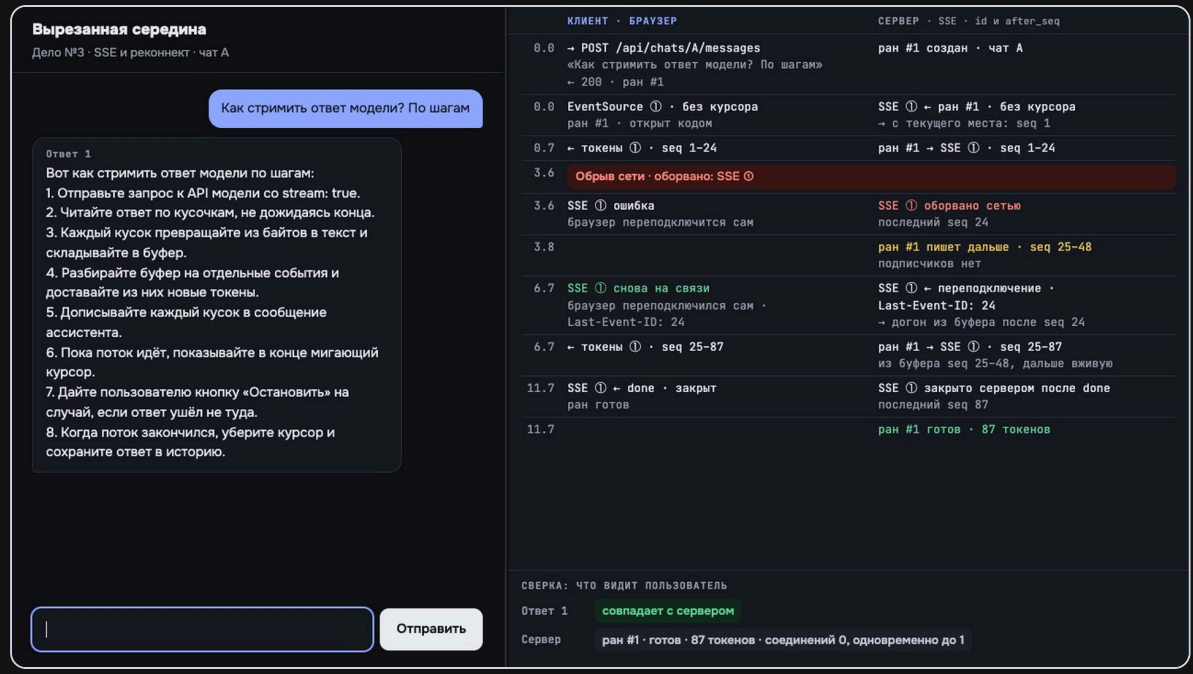
3 КЛИЕНТ ДОЛЖЕН
не мешать, **пока браузер переподключается**: не закрывать EventSource в `onerror` ✓ у нас так

---

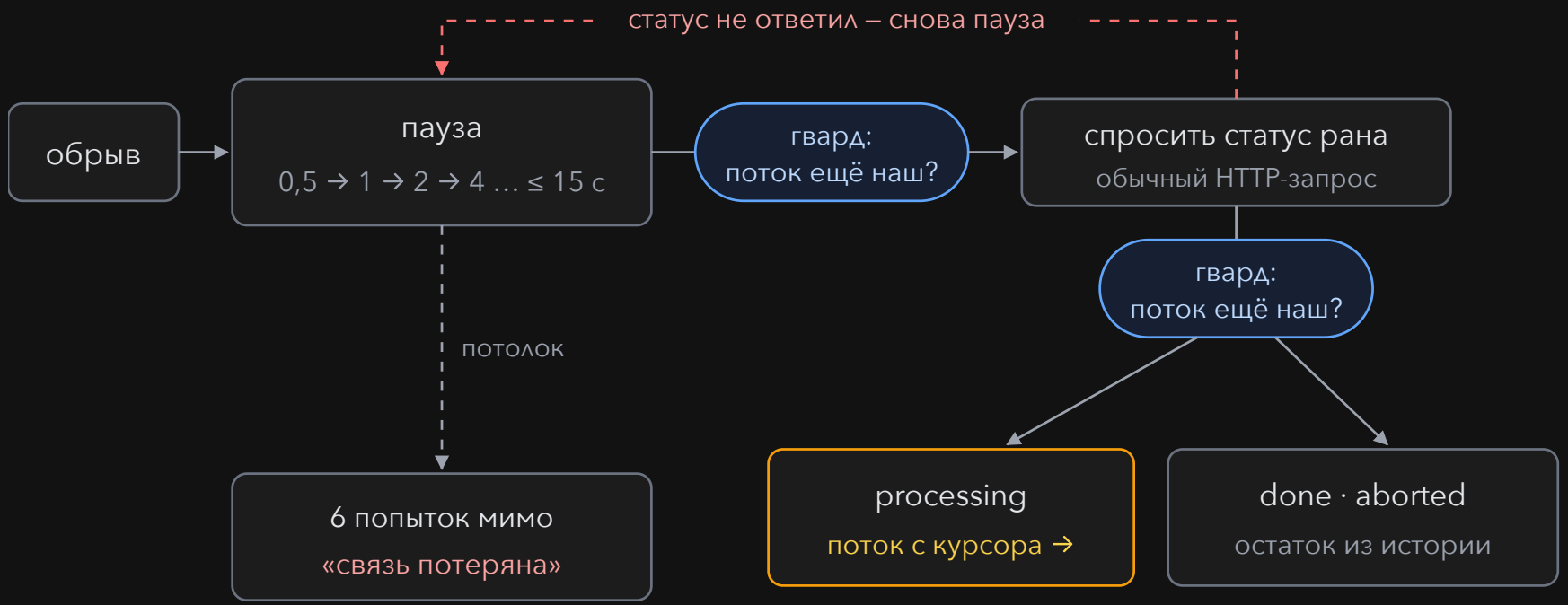
Но на 502 или 401 браузер сдаётся навсегда. **Только тогда** включался наш реконнект – и приходил уже без заголовка.
**Вот почему «не всегда»: дыра – только после ошибки на переподключении.**

# Дело №3 · обрыв без 502

тот же клиент, обычный обрыв — браузер сам приносит курсор



# Фикс: свой реконнект — весь цикл



# Фикс: свой реконнект — поток с курсора

subscribeRun.ts

```
function subscribeRun({ runId, afterSeq, isCurrent, onToken, onEnd, onGiveUp }: SubscribeOptions) {
  let cursor = afterSeq;   // номер последнего токена, который у нас есть
  let attempt = 0;         // попытки переподключения — на схеме
  let es: EventSource;

  function open() {
    es = new EventSource(`/api/runs/${runId}/events?after_seq=${cursor}`);
    es.addEventListener('token', e => {
      if (!isCurrent()) return;      // гвард из дела №1: поток ещё наш?
      const { seq, text } = JSON.parse(e.data);
      if (seq ⩽ cursor) return;
      cursor = seq;
      attempt = 0;
      onToken(seq, text);
    });
    // … done → закрыть поток и onEnd
    es.onerror = () => {
      es.close();
      if (isCurrent()) retry();
    };
  }
```

# Фикс: одна точка входа

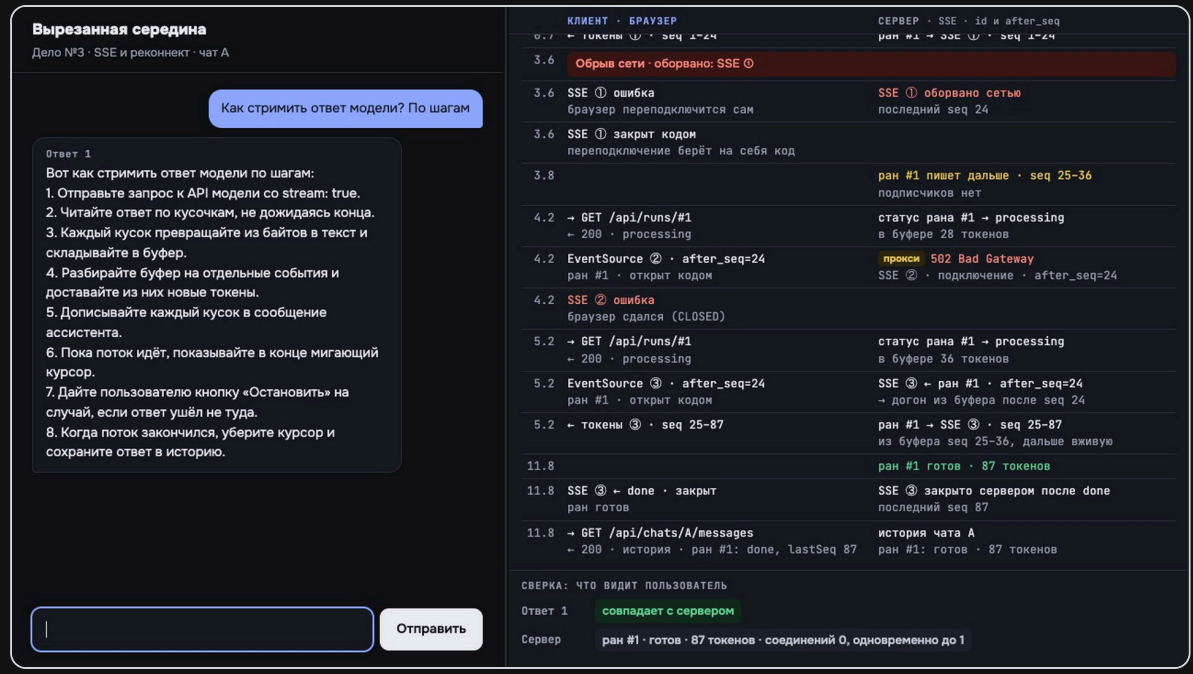
Chat.tsx · экран чата

```
const { data: messages = [], refetch } = api.useGetMessagesQuery(chatId); // история чата
const cache = useChatCache(chatId);        // хелперы кэша, как в деле №2
const generation = useRef(0);              // поколение — гвард из дела №1

const live = messages.findLast(m => m.role === 'assistant' && m.status === 'processing');
useEffect(() => {
  if (!live?.runId) return;                    // живого рана нет — слушать нечего
  const myGen = ++generation.current;
  const stop = subscribeRun({
    runId: live.runId,
    afterSeq: live.lastSeq ?? 0,               // курсор — из истории
    isCurrent: () => myGen === generation.current,
    onToken: (seq, text) => cache.appendToken(live.id, seq, text),
    onEnd: () => refetch(),                    // ран закончился — сверка с историей
    onGiveUp: () => cache.markStalled(live.id),  // «связь потеряна»
  });
  return () => {
    generation.current++;                      // ушли — колбэки молчат
    stop();                                    // и соединение закрыто
  };
}, [live?.runId, chatId]);                     // переподписка — только на новый ран
```

# Дело №3 · фикс применён

обрыв и 502 на переподключении – курсор держит клиент



# Заключение по делу №3

- Сеть виновна в обрыве – **не в потере**
- Виновник – **«Монтажёр»**: наш реконнект без позиции
- Браузер – помощник, а не опора: обычный обрыв лечит `Last-Event-ID`, а 502 и возврат в чат – только свой курсор

ЗАПИСЬ В БЛОКНОТ · ДЕЛО №3

*Соединение вернулось, а генерация ушла вперёд. И никто не сказал, **где** мы остановились.*

**ОПРАВДАНА**

**в обрыве – да, в потере – нет**

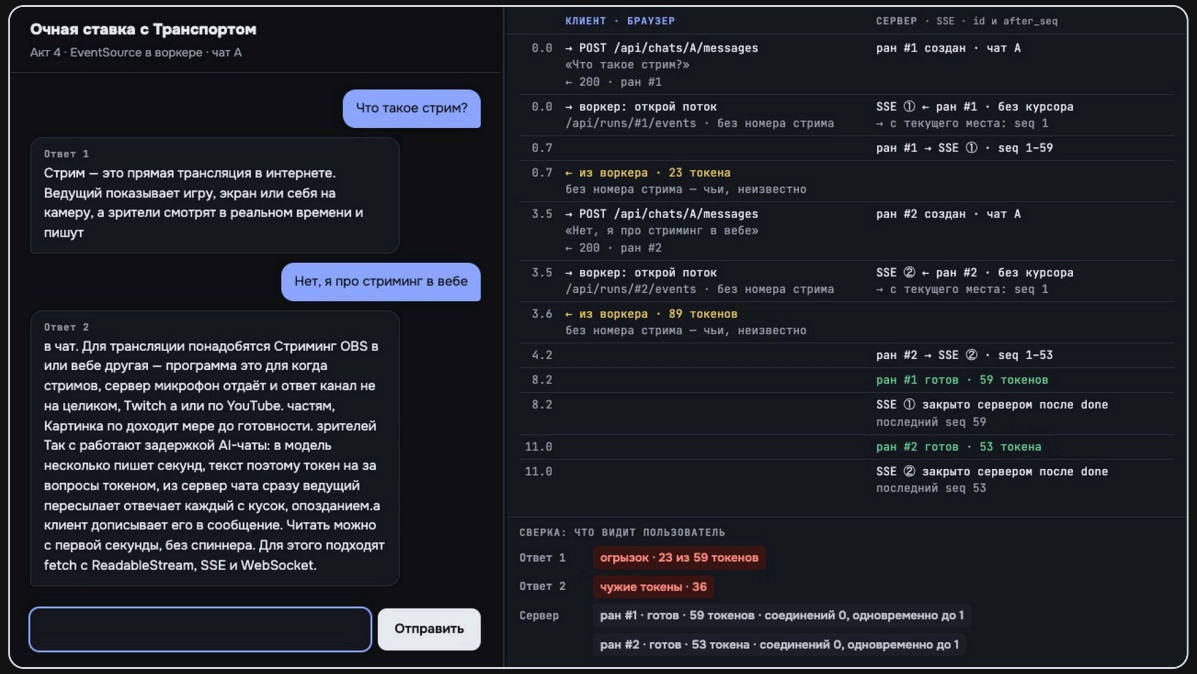
# Очная ставка

## Транспорт

«смените меня – и всё пройдёт»

# Следственный эксперимент

EventSource в воркере · запись в «текущий» ответ · серверная отмена выключена



# Почему воркер не спас

протокол переноса · как было

```
// главный поток
export function openStream(runId: string, appendToCurrent: (text: string) ⇒ void) {
  worker.postMessage({ url: `/api/runs/${runId}/events` });   // «открой» — и всё
  worker.onmessage = e ⇒ appendToCurrent(e.data.text);       // чей токен? пишем в текущий
}
// …
// воркер
self.onmessage = (e: MessageEvent<{ url: string }>) ⇒ {
  const es = new EventSource(e.data.url);
  es.addEventListener('token', ev ⇒ self.postMessage(JSON.parse(ev.data))); // токены — анонимно
  es.addEventListener('done', () ⇒ es.close());
};
```

# Протокол владения

протокол переноса · как стало

```
// главный поток
export function openStream(runId: string, write: (text: string) ⇒ void) {
  if (current) worker.postMessage({ type: 'abort', streamId: current }); // ② отмена
  const streamId = ++current;                          // ① удостоверение
  worker.postMessage({ type: 'open', streamId, url: `/api/runs/${runId}/events` });
  worker.onmessage = e ⇒ {
    if (e.data.streamId === streamId) write(e.data.text); // ① гвард
  };
}
// …
// воркер
if (cmd.type === 'abort') {
  sources.get(cmd.streamId)?.close(); // ② отмена
  sources.delete(cmd.streamId);
  return;
}
// …
es.addEventListener('token', ev ⇒
  self.postMessage({ streamId: cmd.streamId, ...JSON.parse(ev.data) })); // ① с удостоверением
```

# Как выбирать транспорт: два вопроса

Вместо «какой моднее»: **кто владеет соединением · кто платит за реконнект**

ЧЕМ ЕДУТ ТОКЕНЫ – ПРОТОКОЛ

**fetch-стрим** – реконнект, курсор и декодирование – ваши

**SSE** – реконнект и декодирование браузера, но реконнект сдаётся на 502 и 401; курсор – если бэк шлёт `id:`; только GET

**WebSocket** – всё ваше: и протокол, и реконнект, и курсор

ГДЕ ЖИВЁТ СТРИМ – МЕСТО

**Главный поток или воркер** – это адрес, а не владелец: удостоверение и отмену всё равно пишете вы

# Транспорт лечит баги разбора.
# Баги владения — нет.



# Картотека: все оправданы

- **№1 · Замыкание** – видело мир слишком свежим; стало орудием фикса
- **№2 · Кэш** – отдал устаревшее, но по своим правилам прав
- **№3 · Сеть** – виновна в обрыве, не в потере
- **№4 · Транспорт** – лечит баги разбора, но не баги владения

Настоящие виновники – **вне картотеки**.

# Доска следствия

Три дела — одна банда?

# Доска

ДЕЛО №1 · ЧУЖИЕ ТОКЕНЫ

**«Кто последний»**

фикс — удостоверение

ДЕЛО №2 · ПУСТОТА ПРИ ВОЗВРАТЕ

**«Англичанин»**

фикс — синхронизация при возврате

ДЕЛО №3 · ВЫРЕЗАННАЯ СЕРЕДИНА

**«Монтажёр»**

фикс — курсор

Три исполнителя. Три фикса. **Совпадение?**

# Перечитаем блокнот

ЗАПИСЬ В БЛОКНОТ · ДЕЛО №1

*Стрим ещё писал, а адрес уже сменился. И никто не спросил,* **кто** *пишет.*

ЗАПИСЬ В БЛОКНОТ · ДЕЛО №2

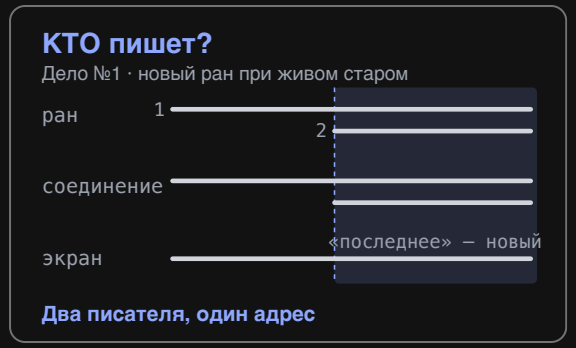
*Мы ушли, а сервер продолжал писать. И никто не сказал кэшу,* **когда** *его снимок устарел.*

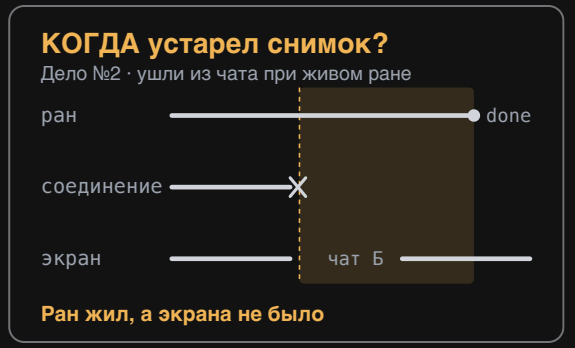
ЗАПИСЬ В БЛОКНОТ · ДЕЛО №3

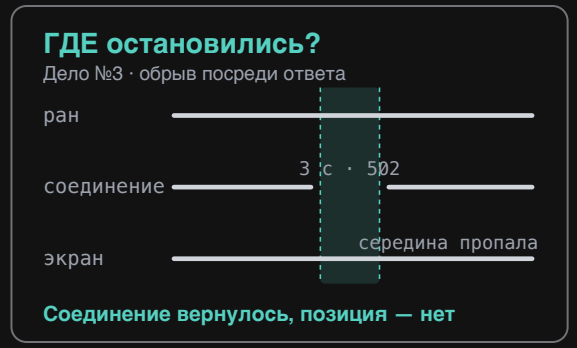
*Соединение вернулось, а генерация ушла вперёд. И никто не сказал,* **где** *мы остановились.*

**Кто.**
**Когда.**
**Где.**

# Три жизни стрима




Каждый — там, где одна жизнь кончилась, а другая ещё нет.

**React видит только нижнюю линию — экран.**

# Почему React сам не заметит: pull и push

### 📬 React – pull

сам решает, **когда посмотреть** на данные

рендерит снимок – и ждёт следующего рендера

*как почтовый ящик: заглядываешь, когда захотел*

### 🔔 Стрим – push

сам решает, **когда прислать** данные

токены приходят между рендерами – и даже после ухода

*как уведомление: приходит, когда пришло*

---

Сами друг другу они ничего не скажут.

**Нужен посредник: принимает push и отдаёт React снимок.**

# Три графы – контракт

### КТО Удостоверение

ДЕЛО №1

- У каждого стрима номер
- Адрес записи – при отправке
- Перед записью – «я ещё текущий?»

### КОГДА Синхронизация

ДЕЛО №2

- Уходя – закрыть и погасить
- Возвращаясь – сверить с сервером

### ГДЕ Курсор

ДЕЛО №3

- Продолжать с позиции
- Свой реконнект: пауза и статус

---

Контракт **двусторонний**: часть графы «где» держит сервер – номера, буфер и статусы.

# У потока — владелец вне рендера

Стрим владеет транспортом.
React подписывается на результат.

Писать токены в стейт можно — только с удостоверением на входе.

Шире, чем LLM: WebSocket · воркеры · совместное редактирование · телеметрия

# Дело закрыто

Три дела закрыты. Виновники названы.



# Спасибо! Ваши вопросы?

📱 Telegram: @AlexRixten 💻 GitHub: AlexRixten